要记得哦 (Ding-Zhe) 前端接口文档

> 版本: v1.0 | 更新日期: 2026-04-24

一、通信约定

1.1 基础配置

项目	值
API 基础路径	`VITE_API_URL` 环境变量，默认 `/api`
后端端口	`3001`（可通过 `PORT` 环境变量配置）
认证方式	Bearer Token (JWT)，Header: `Authorization: Bearer <token>`
Token 存储	`localStorage.getItem('dingzhe_token')`
Token 有效期	30 天

1.2 统一响应格式

```
interface ApiResponse<T> {
  ok: boolean
  data?: T
  error?: {
    code: string       // 错误码，如 'UNAUTHORIZED'
    message: string    // 错误描述
  }
}
```

1.3 全局错误码

错误码	含义
`UNAUTHORIZED`	未授权（缺少 token）
`INVALID_TOKEN`	Token 过期或无效
`INVALID_INPUT`	请求参数不合法
`NOT_FOUND`	资源不存在
`NETWORK_ERROR`	网络请求失败
`INTERNAL_ERROR`	服务器内部错误

1.4 HTTP 客户端

文件：`src/api/client.ts`

```
// JSON 请求
apiFetch<T>(path: string, options?: RequestInit): Promise<ApiResponse<T>>

// 文件上传
apiUpload<T>(path: string, formData: FormData): Promise<ApiResponse<T>>

// Token 管理
setToken(token: string): void
clearToken(): void
```

二、数据模型

2.1 User - 用户

```
interface User {
  id: string
  name: string
  avatar: string              // emoji 或图片 URL
  partnerId: string
  onboarded: boolean
  userCity?: string
  birthday?: string           // "YYYY-MM-DD"
  bio?: string                // 最多 50 字
}
```

2.2 TaskTemplate - 任务模板

```
interface TaskTemplate {
  id: string
  name: string
  category: Category          // 'health' | 'life' | 'work' | 'study' | 'other'
  remindTime: string          // "HH:mm"
  repeatRule: RepeatRule       // 'once' | 'daily' | 'weekly'
  weeklyDays: number[]        // 0=周日, 1=周一 ... 6=周六
  followUpIntensity: FollowUpIntensity  // 'light' | 'standard' | 'strong'
  isActive: boolean
  createdAt: number           // 时间戳
  itemType: ItemType          // 'care' | 'todo' | 'confirm'
  creatorId: string
  receiverId: string
  note: string
}
```

2.3 TaskInstance - 任务实例

```
interface TaskInstance {
  id: string
  templateId: string
  scheduledTime: number
  status: InstanceStatus       // 'draft' | 'pending' | 'awaiting' | 'deferred' | 'completed' | 'skipped' | 'exp
  followUpCount: number
  maxFollowUps: number
  followUpInterval: number
  nextFollowUpAt: number | null
  deferredSince: number | null
  completedAt: number | null
```

```
  skippedAt: number | null
  expiredAt: number | null
  actionLog: ActionLog[]
  relationStatus: RelationStatus  // 'draft' | 'sent' | 'delivered' | 'seen' | 'responded' | 'resolved'
  feedback: string | null
}

interface ActionLog {
  timestamp: number
  action: 'reminded' | 'user_completed' | 'user_deferred' | 'user_skipped' | 'auto_deferred' | 'follow_up_sent'
  note: string
}
```

2.4 FeelingEntry - 感受记录

```
interface FeelingEntry {
  id: string
  userId: string
  content: string
  mood: string                // emoji
  entryType?: FeelingEntryType  // 'text' | 'photo' | 'mood' | 'reminder'
  aboutPartnerId?: string
  photoUrl?: string           // 向后兼容：首张照片
  photoUrls?: string[]        // 多张照片（最多 9 张）
  linkedTaskId?: string
  createdAt: number
  isDraft: boolean
  isHidden?: boolean
  hiddenPhotoIndices?: number[]
  location?: { lat: number; lng: number; name?: string }
  mediaTypes?: ('image' | 'video')[]
  likedBy?: string[]          // 点赞用户 ID 数组
}
```

2.5 Comment - 评论

```
interface Comment {
  id: string
  entryId: string
  content: string
  author: 'user' | 'ai'
  userId?: string
  createdAt: number
}
```

2.6 RelationshipSpace - 关系空间

```
interface RelationshipSpace {
  id: string
  userIds: [string, string]
  relationType: RelationType   // 'self' | 'couple' | 'family' | 'friends'
  companion: CompanionAnimal   // 'cat' | 'dog' | 'bear' | 'rabbit' | 'penguin' | 'fox'
  createdAt: number
  petState: PetState
  anniversaries: Anniversary[]
}

interface PetState {
  mood: PetMood               // 'happy' | 'content' | 'neutral' | 'lonely' | 'sleepy'
  energy: number              // 0-100
  lastFed: number | null
```

```
  lastPetted: number | null
  todayInteractions: number
  interactionDate: string     // "YYYY-MM-DD"
}

interface Anniversary {
  id: string
  title: string
  date: string                // "MM-DD"
  year: number | null
  emoji: string
  isRecurring: boolean
  isPrimary: boolean
}
```

2.7 NarrativeEntry - 叙事条目

```
interface NarrativeEntry {
  id: string
  userId: string
  title: string
  bodyText: string
  petSummary: string
  photoUrls: string[]
  feelingIds: string[]
  createdAt: number
}
```

2.8 RelayMessage - 情感代传

```
interface RelayMessage {
  id: string
  senderId: string
  receiverId: string
  originalText: string
  selectedVersion: RelayVersionType  // 'gentle' | 'casual' | 'direct'
  relayText: string
  status: RelayMessageStatus         // 'sent' | 'delivered' | 'read'
  createdAt: number
  readAt: number | null
}
```

2.9 ChatMessage - 宠物对话

```
interface ChatMessage {
  id: string
  role: ChatRole              // 'user' | 'cat' | 'system'
  content: string
  timestamp: number
  expression?: PetExpression
  actionType?: ChatQuickAction  // 'pet' | 'play' | 'advice'
  isRelay?: boolean
  relayId?: string
  taskCreated?: { name: string; time: string; receiverName: string }
}
```

2.10 Notification - 通知

```
interface Notification {
  id: string
  toUserId: string
  message: string
  relatedTemplateId?: string
  timestamp: number
  read: boolean
}
```

2.11 NLP 解析结果

```
interface ParsedTask {
  name: string
  time: string | null
  dateContext: 'today' | 'tomorrow' | 'day-after' | 'specific' | null
  specificDate: string | null
  repeatRule: RepeatRule | null
  weeklyDays: number[]
  category: Category | null
  confidence: { time: boolean; repeat: boolean; category: boolean }
  receiver: string | null
  itemType: ItemType | null
}
```

三、REST API 接口

3.1 认证模块 `/auth`

文件：`src/api/auth.ts` | `server/src/routes/auth.ts`

POST /auth/login — 昵称登录

```
请求体: { nickname: string }
响应体: { token: string; user: UserProfile }
认证: 否
```

POST /auth/login-phone — 手机号登录

```
请求体: { phone: string; password: string }
响应体: { token: string; user: UserProfile }
认证: 否
```

POST /auth/register — 注册

```
请求体: { phone: string; code: string; password: string }
响应体: { token: string; user: UserProfile }
认证: 否
```

POST /auth/send-code — 发送验证码

```
请求体: { phone: string; purpose: 'register' | 'reset_password' }
响应体: { message: string }
认证: 否
```

POST /auth/reset-password — 重置密码

```
请求体: { phone: string; code: string; newPassword: string }
响应体: { token: string; user: UserProfile }
认证: 否
```

GET /user/me — 获取当前用户信息

```
响应体: UserProfile
认证: 是
```

UserProfile:

```
interface UserProfile {
  id: string
  nickname: string
  avatar: string
  mode: 'single' | 'dual'
  partnerId: string | null
  onboarded: boolean
  createdAt: string
  partner?: { id: string; nickname: string; avatar: string } | null
}
```

3.2 任务模块 `/task`

文件: `src/api/tasks.ts` | `server/src/routes/task.ts`

GET /task/list — 获取任务列表

```
响应体: TaskTemplate[] (含 instances 子数组)
认证: 是
```

POST /task/create — 创建任务

```
请求体: CreateTaskInput
响应体: TaskTemplate
认证: 是
```

```
interface CreateTaskInput {
  name: string
  category: Category
```

```
  remindTime: string          // "HH:mm"
  repeatRule: RepeatRule
  weeklyDays: number[]
  followUpIntensity: FollowUpIntensity
  itemType: ItemType
  receiverId: string
  note: string
  scheduledDateOffset?: number | null
  specificDate?: string | null
}
```

POST /task/update_status — 更新任务状态

```
请求体: { instanceId: string; status: InstanceStatus; feedback?: string }
响应体: TaskInstance
认证: 是
```

3.3 照片模块 `/photo`

文件: `src/api/photos.ts` | `server/src/routes/photo.ts`

POST /photo/upload — 上传照片

```
请求体: FormData { file: File; description?: string; relatedTaskId?: string; mode?: string; tags?: string[] }
响应体: Photo
认证: 是
限制: 最大 5MB
```

```
interface Photo {
  id: string
  userId: string
  mode: string
  url: string                 // /uploads/{filename}
  description: string | null
  relatedTaskId: string | null
  tags: string[]
  createdAt: string
  mediaType?: 'image' | 'video'
}
```

GET /photo/list — 获取照片列表

```
参数: ?limit=50
响应体: Photo[]
认证: 是
```

3.4 空间模块 `/space`

文件: `src/api/space.ts` | `server/src/routes/space.ts`

GET /space/my — 获取用户空间

```
响应体: RelationshipSpace
认证: 是
```

POST /space/companion — 更新宠物

```
请求体: { companion: CompanionAnimal }
响应体: RelationshipSpace
认证: 是
```

GET /space/anniversaries — 获取纪念日列表

```
响应体: Anniversary[]
认证: 是
```

POST /space/anniversaries — 创建纪念日

```
请求体: Omit<Anniversary, 'id'>
响应体: Anniversary
认证: 是
```

PATCH /anniversaries/:id — 更新纪念日

```
请求体: Partial<Omit<Anniversary, 'id'>>
响应体: Anniversary
认证: 是
```

DELETE /anniversaries/:id — 删除纪念日

```
响应体: { ok: true }
认证: 是
```

POST /space/pet-interact — 宠物互动

```
请求体: { type: 'pet' | 'feed' }
响应体: PetState
认证: 是
```

3.5 关系模块 `/relation`

文件: `server/src/routes/relation.ts`

POST /relation/invite — 邀请伴侣

```
请求体: { targetNickname: string }
认证: 是
错误码: USER_NOT_FOUND | SELF_INVITE | ALREADY_BOUND | PARTNER_BOUND
```

POST /relation/dissolve — 解除关系

```
请求体: 无
响应体: { mode: string }
认证: 是
```

3.6 宠物聊天模块 `/pet-chat`

文件：`src/api/pet-chat.ts` | `server/src/routes/pet-chat.ts`

超时：12000ms | 降级：服务端不可用时客户端直连 Kimi API

POST /pet-chat/reply — 获取宠物回复

```
认证: 否
```

请求体:

```typescript
interface PetChatRequest {
  message: string
  history: Array<{ role: 'user' | 'assistant'; content: string }>  // 最多 10 条
  context: {
    companionName: string
    mood: string
    energy: number
    relationDays: number
    userCity?: string
    userName?: string
    partnerName?: string
    partnerCity?: string
    upcomingAnniversaries?: Array<{ title: string; emoji: string; daysUntil: number; isToday: boolean }>
    taskSummary?: {
      pendingTasks: Array<{ name: string; category: string; itemType: string; forPartner: boolean }>
      todayCompleted: number
      todayTotal: number
      overdueCount: number
    }
  }
  imageUrls?: string[]        // 仅 http/https
}
```

响应体:

```typescript
{ text: string; fallback: boolean }
```

3.7 情感代传模块 `/emotion-relay`

文件：`src/api/emotion-relay.ts` | `server/src/routes/emotion-relay.ts`

超时：15000ms

POST /emotion-relay/generate — 生成多种表达版本

```
认证：否
```

请求体:

```
{
  message: string
  context: {
    senderName: string
    receiverName: string
    companionName: string
    relationDays: number
    relationType: string
  }
}
```

响应体:

```
{ gentle: string; casual: string; direct: string; fallback: boolean }
```

POST /emotion-relay/generate-smart — 生成最优表达

```
认证：否
```

响应体:

```
{ text: string; tone: 'gentle' | 'casual' | 'direct'; fallback: boolean }
```

3.8 叙事生成模块 `/narrative`

文件：`src/api/narrative.ts` | `server/src/routes/narrative.ts`

POST /narrative/generate — 生成叙事

```
认证：是
```

请求体:

```
{
  scope: 'self' | 'relationship'
  partnerId?: string
  photoId?: string
  taskIds?: string[]
  feelings?: Array<{ content: string; mood: string; photoCount: number }>
  photoUrls?: string[]
}
```

响应体:

```
{
  id: string
  scope: string
  userId: string
  partnerId: string | null
  content: string
  createdAt: string
}
```

四、Store API（前端状态管理）

文件: `src/store.ts` | 类型: `export type Store = ReturnType<typeof useStore>`

4.1 状态数据

字段	类型	说明
`users`	`User[]`	用户列表
`space`	`RelationshipSpace`	关系空间
`templates`	`TaskTemplate[]`	任务模板
`instances`	`TaskInstance[]`	任务实例
`feelings`	`FeelingEntry[]`	感受记录
`comments`	`Comment[]`	评论列表
`narratives`	`NarrativeEntry[]`	叙事条目
`relayMessages`	`RelayMessage[]`	情感代传消息
`notifications`	`Notification[]`	通知列表
`toast`	`{ message; type } | null`	Toast 提示
`isApiLoading`	`boolean`	API 加载状态

4.2 派生数据

字段	类型	说明
`deferred`	`TaskInstance[]`	已延迟任务

字段	类型	说明
`awaiting`	`TaskInstance[]`	等待中任务
`pending`	`TaskInstance[]`	未完成任务
`todayDone`	`TaskInstance[]`	今日已完成
`currentPetState`	`PetState`	当前宠物状态
`todayCompletedCount`	`number`	今日完成数
`todayTotalCount`	`number`	今日总数
`todayCareCount`	`number`	今日关心完成数
`relationDays`	`number`	关系天数
`todayAnniversaries`	`Anniversary[]`	今日纪念日
`upcomingAnniversaries`	`Array<{...}>`	即将到来的纪念日
`todayFeelingCount`	`number`	今日感受数
`recentPhotos`	`string[]`	最近照片

4.3 查询方法

```
getTemplate(templateId: string): TaskTemplate | undefined
getReceivedItems(userId: string): TaskInstance[]
getSentItems(userId: string): TaskInstance[]
getDraftItems(userId: string): TaskInstance[]
getUserNotifications(userId: string): Notification[]
getUserProfile(userId: string): User
getFeelings(userId: string, options?: {
  aboutPartner?: boolean
  entryType?: FeelingEntryType
  includeHidden?: boolean
}): FeelingEntry[]
getComments(entryId: string): Comment[]
getPetComment(feelingId: string): string | null
getNarrative(id: string): NarrativeEntry | null
getNarratives(userId: string): NarrativeEntry[]
getAiTaskSummary(userId: string): {
  pendingTasks: Array<{ name; category; itemType; forPartner }>
  todayCompleted: number
  todayTotal: number
  overdueCount: number
}
getUnreadRelays(userId: string): RelayMessage[]
```

4.4 任务操作

```
createTask(data: CreateTaskInput): void
saveDraftTask(data: CreateTaskInput): void
promoteDraftToSent(templateId: string): void
deleteTemplate(templateId: string): void
updateTemplate(templateId: string, patch: Partial<Pick<TaskTemplate, 'name' | 'remindTime' | 'repeatRule' | 'wee
updateInstanceDate(instanceId: string, newDate: Date): void
completeInstance(instanceId: string): void
deferInstance(instanceId: string, delayMinutes: number): void
skipInstance(instanceId: string): void
```

```
cantDoInstance(instanceId: string): void
respondWithFeedback(instanceId: string, feedbackText: string): void
markSeen(instanceId: string): void
```

4.5 感受操作

```
saveFeeling(content: string, mood: string, aboutPartnerId?: string, entryType?: FeelingEntryType, photoUrls?: st
updateFeeling(feelingId: string, patch: Partial<Pick<FeelingEntry, 'content' | 'mood' | 'location' | 'photoUrls'
deleteFeeling(feelingId: string): void
convertFeelingToTask(feelingId: string): void
toggleHideFeeling(feelingId: string): void
toggleLikeFeeling(feelingId: string, userId: string): void
toggleHidePhoto(feelingId: string, photoIndex: number): void
addComment(entryId: string, content: string, author: 'user' | 'ai', userId?: string): void
generateNarrativeEntry(feelingIds?: string[]): Promise<NarrativeEntry | null>
```

4.6 用户与空间

```
updateUserProfile(userId: string, patch: Partial<Pick<User, 'name' | 'avatar' | 'userCity' | 'birthday' | 'bio'>
completeOnboarding(userId: string): void
resetOnboarding(userId: string): void
bindPartner(userId: string, partnerId: string): void
dissolveRelationship(): Promise<void>
setUserCity(userId: string, city: string): void
updateSpaceCompanion(companion: CompanionAnimal): void
updateSpaceRelationType(relationType: RelationType): void
petInteraction(type: 'pet' | 'feed'): void
```

4.7 纪念日

```
addAnniversary(a: Omit<Anniversary, 'id'>): void
removeAnniversary(id: string): void
updateAnniversary(id: string, patch: Partial<Omit<Anniversary, 'id'>>): void
```

4.8 通知与消息

```
addNotification(toUserId: string, message: string, relatedTemplateId?: string): void
dismissNotification(notifId: string): void
sendRelay(senderId: string, receiverId: string, originalText: string, selectedVersion: RelayVersionType, relayTe
markRelayRead(relayId: string): void
showToast(message: string, type?: 'success' | 'info' | 'skip'): void
```

五、本地持久化

文件：`src/lib/storage.ts`

localStorage Key	数据类型
`dingzhe_users`	`User[]`

localStorage Key	数据类型
`dingzhe_space`	`RelationshipSpace`
`dingzhe_templates`	`TaskTemplate[]`
`dingzhe_instances`	`TaskInstance[]`
`dingzhe_feelings`	`FeelingEntry[]`
`dingzhe_comments`	`Comment[]`
`dingzhe_narratives`	`NarrativeEntry[]`
`dingzhe_relay_messages`	`RelayMessage[]`
`dingzhe_token`	`string`（JWT Token）

自动同步：store 中任一状态变化时通过 `useEffect` 自动写入 localStorage。

六、API 路由总览

```
/api
├── auth/
│   ├── POST   login              昵称登录
│   ├── POST   login-phone        手机号登录
│   ├── POST   register           注册
│   ├── POST   send-code          发送验证码
│   └── POST   reset-password     重置密码
├── user/
│   └── GET    me           [JWT] 获取当前用户
├── task/
│   ├── GET    list         [JWT] 获取任务列表
│   ├── POST   create       [JWT] 创建任务
│   └── POST   update_status [JWT] 更新任务状态
├── photo/
│   ├── POST   upload       [JWT] 上传照片（FormData, 5MB）
│   └── GET    list         [JWT] 获取照片列表
├── space/
│   ├── GET    my           [JWT] 获取用户空间
│   ├── POST   companion    [JWT] 更新宠物
│   ├── GET    anniversaries [JWT] 获取纪念日
│   ├── POST   anniversaries [JWT] 创建纪念日
│   └── POST   pet-interact [JWT] 宠物互动
├── anniversaries/
│   ├── PATCH  :id          [JWT] 更新纪念日
│   └── DELETE :id          [JWT] 删除纪念日
├── relation/
│   ├── POST   invite       [JWT] 邀请伴侣
│   └── POST   dissolve     [JWT] 解除关系
├── pet-chat/
│   └── POST   reply              宠物聊天（超时 12s）
├── emotion-relay/
│   ├── POST   generate           生成多版本表达（超时 15s）
│   └── POST   generate-smart     生成最优表达（超时 15s）
└── narrative/
    └── POST   generate     [JWT] 生成叙事
```

七、第三方服务

服务	用途	配置项
Kimi K2.5 (Moonshot)	宠物聊天 / 情感代传 / 叙事生成	`KIMI_API_KEY`, `KIMI_BASE_URL`
和风天气 (QWeather)	宠物聊天中的天气信息	`QWEATHER_API_KEY`, `QWEATHER_BASE_URL`